

Programmieren 2 - 3. Test - Vorbereitung B Sommersemester 2026

Allgemeine Hinweise:

Elektronische Geräte sind abzuschalten und schwer erreichbar (z.B. in einer verschlossenen Tasche, aber **nicht** am Körper) zu verstauen (zur Not am Vortragendenpult hinterlegen).

Die Prüfungsaufgaben befinden sich auf der Rückseite der Angabe.

Vorbereitung:

Die folgenden Aufgaben sind in eclipse zu lösen. Öffnen Sie eclipse im Menü links oben (Applications - Development - Eclipse). Kopieren Sie die Dateien Main.java und Checks_Do_Not_Change.java in Ihr Java Projekt. Ändern Sie den Namen des Packages (`package` Anweisung) in Main und in Checks_Do_Not_Change auf Ihren package Namen (`a+Matrikelnummer`).

Erlaubte Hilfsmittel:

Zugriff auf die Webseite der Lehrveranstaltung und auf die Java Dokumentation (<https://docs.oracle.com/en/java/javase/17/docs/api>) ist erlaubt. Andere Webseiten dürfen nicht zugegriffen werden. Es dürfen neben dem Webbrowser und PDF-Viewer nur Programme verwendet werden, die für die Lösung der Aufgabenstellung notwendig sind (Eclipse und Terminal). Als Java-Referenz kann auch das in Eclipse integrierte Hilfe-System verwendet werden.

Die Bearbeitungszeit beträgt 90 Minuten. Alle Instanzvariablen sind `private` zu deklarieren. Veränderungen an den von der LV-Leitung vorgegebenen Klassen sind (soweit nicht explizit in der Aufgabenstellung verlangt) nicht erlaubt. Es werden ausschließlich allgemeingültige Lösungen für Probleme akzeptiert. Die Verwendung von `instanceof`, `getClass`, `isAssignable` etc. zur Lösung der Aufgabenstellungen führt zu Punktabzug. Die Abgabe muss kompilieren. Es werden nur die Aufgaben bewertet die in der main Methode der Main-Klasse dekommentiert wurden.

Sie dürfen bei allen Aufgaben davon ausgehen, dass alle übergebenen Parameter sinnvolle Werte sind und keine null-Objekte auftreten.

1. (1 Punkt) Komplettieren Sie die in der Klasse `Main` vordefinierte Klassenmethode `Potion[] sortPotions(List<Potion> potions)` so, dass die Liste `potions` in zwei Teile `A`, und `B` aufgeteilt wird. `A` enthält alle Objekte aus `potions`, deren Gewicht (`getWeight`) größer gleich dem Durchschnittsgewicht aller Objekte aus `potions` ist, absteigend nach Preis (`getPrice`) sortiert. `B` enthält alle Objekte aus `potions`, deren Gewicht kleiner als das Durchschnittsgewicht aller Objekte aus `potions` ist, aufsteigend nach Preis sortiert.
Die beiden Teillisten werden dann zum Füllen eines Arrays verwendet, wobei aus jeder Teilliste immer jeweils ein Element aus `A` und eines aus `B` entnommen wird und abwechselnd in der Reihenfolge `AB` gefolgt von der Reihenfolge `BA` in das Array eingefügt wird (es ergibt sich also insgesamt eine Reihenfolge `ABBAABBAAB...`). Das Verfahren endet, sobald alle Elemente aus einer der beiden Teillisten abgearbeitet sind. Eventuell nicht bearbeitete Elemente in der längeren Liste werden ignoriert. Das so gefüllte Array, dessen Größe exakt der Anzahl der enthaltenen Objekte entsprechen muss, ist zu retournieren. Es bleibt Ihnen überlassen, welche Vorgangsweise für das Sortieren der Werte Sie wählen.
2. (1 Punkt) Komplettieren Sie die in der Klasse `Main` vordefinierte Klassenmethode `TreeMap<Integer, Integer> mapItemsPerPriceCategory(List<MagicItem> items)` so, dass eine `TreeMap` retourniert wird, die jeder in `items` auftretenden Preiskategorie als Schlüssel, die Summe der Gewichte der `MagicItem`-Objekte aus `items` in der entsprechenden Kategorie als Wert zuordnet. Die Sortierung der Schlüsselwerte in der Map muss absteigend erfolgen. Die Preiskategorie ergibt sich, als Wert der letzten beiden Ziffern des Preises (z. B. Preis 29, 129 oder 7429 ergibt Preiskategorie 29, Preis 1, 701 oder 7801 ergibt Preiskategorie 1 usw.).
3. (1 Punkt) Erstellen Sie eine Klasse `HazardousSpell`, die von `AttackingSpell` erbt. Die Methode `cast` ist so zu override, dass prinzipiell dasselbe passiert wie in der Basisklasse, das Aussprechen des Zaubers kostet aber dem ausübenden Objekt (`source` eine Anzahl von Lebenspunkten (HP), die mit jeder Anwendung des Zaubers beginnend bei 10 Punkten um 10 Punkte ansteigt. (Also das erste Mal kostet 10 HP, das zweite Mal 20 HP usw.). Hat das ausübende Objekt nicht genügend Lebenspunkte, um die Aktion zu überleben, wird der Zauber abgebrochen und weder wird `cast` aufgerufen, noch werden die Kosten für den nächsten Aufruf erhöht.
Um Lebenspunkte bei `source` zu prüfen oder abzuziehen, dürfen Sie davon ausgehen, dass es sich bei `source` im Kontext dieser Aufgabe um ein `Wizard`-Objekt handelt. Der Typecast `((Wizard)source)` ist also immer erfolgreich. Sie werden einen Konstruktor für die Klasse benötigen, der dieselben Parameter hat, wie die Basisklasse und einfach alle Parameter an den Konstruktor der Basisklasse weiterreicht. Eventuell benötigen Sie zusätzliche Getter- oder Setter-Methoden.
4. (1 Punkt) Erstellen Sie eine public Methode `boolean checkName(int minCount)` in der Klasse `Potion`, die genau dann `true` retourniert, wenn im Namen des `this`-Objekts zumindest ein Zeichen mindestens `minCount` mal auftritt. (Groß- und Kleinschreibung werden nicht unterschieden, also 'A' und 'a' werden z. B. gemeinsam gezählt. Um auf den Namen zugreifen zu können, benötigen Sie wahrscheinlich auch eine getter-Methode für den Namen in der Klasse `MagicItem`. Zur Umwandlung zwischen Groß- und Kleinbuchstaben können die Klassenmethoden `Character.toLowerCase(char c)` bzw. `Character.toUpperCase(char c)` eingesetzt werden. Hinweis falls Sie mit Streams arbeiten wollen: Um aus dem Namen einen `Stream<Character>` zu erstellen, können Sie `getName().chars().mapToObj(i->(char)i)` verwenden.)
Komplettieren Sie die in `Main` vordefinierte Methode `selectPotions(List<Potion> list)` so, dass eine Liste aller `Potion`-Objekte in `list`, für die der Aufruf von `Checks.Do_Not_Change.doSelect true` ergibt, retourniert wird. Die relative Reihenfolge der Objekte ist beizubehalten. `doSelect` verlangt einen Parameter, der das in Java vordefinierte funktionale Interface `Function<Integer, Boolean>` mit einer abstrakten Methode `Boolean apply(Integer value)` implementiert. Sie rufen `Checks.Do_Not_Change.doSelect` für `Potion`-Objekte auf, die das Interface `Function<Integer, Boolean>` anbieten und in denen die Methode `apply` so overridden ist, dass `checkName` mit demselben Parameterwert aufgerufen wird. Das kann, da es sich bei `Function` um ein functional Interface handelt, durch Implementierung des Interfaces in einer geeigneten Klasse, durch Verwenden einer anonymen Klasse oder auch durch einen Lambda-Ausdruck bzw. eine Methoden-Referenz erfolgen.